

15-Day Sprint · Google Interview Ready

For absolute beginners · Industry-aligned · Pattern-first approach

10 Core Patterns	100+ LeetCode Refs	15 Day Plan	5 Interview Tips
-----------------------------	-----------------------------------	--------------------	-----------------------------

■ *Reality check: 15 days is a sprint, not a transformation. This guide maximizes your prep window — but set realistic expectations for a Google interview.*

Prepared: May 2025 | Target: Google SWE Interview

■ Table of Contents

#	Section	Page
1	Reality Check — What Google Actually Expects	3
2	How to Read DSA Problems (The Pattern Framework)	4
3	Pattern 1 — Sliding Window	5
4	Pattern 2 — Two Pointers	7
5	Pattern 3 — Hashing	9
6	Pattern 4 — Recursion & Backtracking	11
7	Pattern 5 — Divide & Conquer	13
8	Pattern 6 — Greedy	14
9	Pattern 7 — Dynamic Programming	15
10	Pattern 8 — Binary Search	18
11	Pattern 9 — Graph Traversal (BFS/DFS)	20
12	Pattern 10 — Stack & Queue	22
13	Bonus Patterns (Prefix Sum, Heap, Union Find, Bit Manip)	24
14	15-Day Study Plan	26
15	How to Respond in a Google Interview	29
16	Curated LeetCode Problem List (Beginner → Advanced)	31
17	Free Resources & References	33

1 — Reality Check: What Google Actually Expects

What Google interviews look like

Google's SWE interviews (L3/L4 entry-level) consist of 4-5 coding rounds + 1 behavioral (Googlyness) round. Each coding round is 45-60 minutes, 1-2 problems, on a shared doc (no IDE, no autocomplete). You are expected to talk out loud the entire time, explain your thought process, analyze complexity, and catch your own bugs.

Expectation	Beginner Assumption	Reality
Difficulty	Easy / beginner friendly	~60% Medium, ~20% Hard, ~20% Easy
Time	I'll figure it out in interview	You need pattern reflexes — muscle memory
Communication	Just write the code	Talking through approach = 40% of score
Complexity	Get it working first	$O(n)$ vs $O(n^2)$ analysis expected always
Edge Cases	Handle the happy path	Null, empty, duplicates — all expected
Topics	Arrays and strings only	Trees, Graphs, DP, BFS/DFS frequently appear

■ 15 days is enough to: learn all 10 patterns, solve ~50 problems with understanding, and develop interview communication habits. It is NOT enough to: hit LeetCode Hard consistently, or reach the top 10% of candidates. Go in knowing this.

What India-based Google interviews specifically test (2024-25)

Based on recent Glassdoor, LeetCode Discuss, and InterviewBit reports for Google India (Hyderabad/Bangalore), the most frequent topics are:

Topic	Frequency	Difficulty Expected
Arrays & Strings	Very High	Easy–Medium
Trees (BFS/DFS)	High	Medium
Dynamic Programming	High	Medium–Hard
Graphs	Medium	Medium
Sliding Window	Medium	Easy–Medium
Binary Search	Medium	Medium
Heaps & Priority Queue	Medium	Medium
Hashing	High	Easy–Medium
Two Pointers	High	Easy–Medium
Backtracking	Low–Medium	Hard

2 — How to Read DSA Problems: The Pattern Framework

The single most important skill in DSA interviews is pattern recognition — seeing a new problem and instantly mapping it to a known technique. This is built through repetition, not intelligence.

The UCRCA Framework (Use this in every problem)

Step	What To Do	Example
U — Understand	Read 2x. Ask clarifying questions. Restate the problem.	"Find the longest substring without repeating characters?"
C — Constraints	Check input size. This determines your target time & space. $n \leq 10^5$ → $O(n \log n)$ or better. $n \leq 10$ → $O(2^n)$.	
R — Recognize	Map keywords to patterns. See table below.	"Longest subarray" → Sliding Window
C — Code	Write clean, commented pseudocode. Write steps in plain English before typing.	
A — Analyze	State time & space complexity. Find edge cases. $O(k)$ space. Edge: empty array?	

Keyword → Pattern Mapping (Your Cheat Sheet)

If you see this keyword...	Think this pattern
longest / shortest / minimum / maximum + subarray/substring	Sliding Window
sorted array + pairs / triplets / sum	Two Pointers
duplicates / frequency / appeared before / count	Hash Map or Set
all combinations / all permutations / all subsets	Backtracking
optimal / min cost / max profit + overlapping subproblems	Dynamic Programming
find position / first occurrence / rotated sorted	Binary Search
shortest path / level by level / nearest neighbor	BFS (Graph)
connected components / cycle detection / DFS	DFS (Graph)
next greater element / matching brackets / undo last	Stack
process in order / sliding window with queue / BFS queue	Queue / Deque
k largest / k smallest / top k frequent	Heap (Priority Queue)
range sum query / prefix sum	Prefix Sum Array

Pattern 1 — Sliding Window

A window is a contiguous range [left, right] over an array or string. You expand right to grow the window and shrink left when a condition breaks. This avoids the $O(n^2)$ brute force of checking every subarray.

When to use

Keywords: longest, shortest, minimum, maximum, subarray, substring, contiguous. The answer involves a contiguous portion of input. You want $O(n)$.

Template (Fixed Window)

```
left = 0; right = 0
while right < n:
    add arr[right] to window
    if window size > k: remove arr[left]; left++
    update answer
    right++
```

Template (Variable Window)

```
left = 0
for right in range(n):
    add arr[right] to window (update freq/sum)
    while window is invalid:
        remove arr[left]; left++
    answer = max(answer, right - left + 1)
```

Complexity

Time: $O(n)$ — each element enters and exits the window at most once. Space: $O(k)$ for the window state.

LeetCode Problems — Sliding Window

#	Problem	Difficulty	Key Insight
121	Best Time to Buy and Sell Stock	Easy	Track min price seen so far
3	Longest Substring Without Repeating Chars	Medium	Hash set to detect duplicates
76	Minimum Window Substring	Hard	Count chars needed; shrink when satisfied
424	Longest Repeating Character Replacement	Medium	Track max freq char; others must be $\leq k$
567	Permutation in String	Medium	Fixed window = $\text{len}(s1)$
643	Maximum Average Subarray I	Easy	Fixed window sum
1004	Max Consecutive Ones III	Medium	Count zeros flipped $\leq k$
239	Sliding Window Maximum	Hard	Use deque for $O(n)$

■ Interview Tip: When you say 'Sliding Window', immediately follow with: 'I'll maintain a window [left, right] and expand right, shrinking left when the condition breaks. This gives me $O(n)$ instead of $O(n^2)$ brute force.'

Pattern 2 — Two Pointers

Two Pointers places one pointer at each end of a sorted array (or both at start for fast/slow). Moving them toward each other eliminates possibilities in $O(1)$ per step, achieving $O(n)$ total.

Types

Type	Setup	Use Case
Opposite ends	left=0, right=n-1 moving inward	Pair sum, palindrome, container
Same direction (fast/slow)	slow=0, fast=0 or fast=1	Linked list cycle, remove duplicates
Partition	left tracks 'valid zone'	Sort colors, move zeros

Template

```
# Opposite-ends pattern
left, right = 0, len(arr) - 1
while left < right:
    current = arr[left] + arr[right]
    if current == target: record; left++; right--
    elif current < target: left++ # need bigger sum
    else: right-- # need smaller sum
```

LeetCode Problems — Two Pointers

#	Problem	Difficulty	Key Insight
167	Two Sum II (sorted)	Easy	Opposite ends, move based on sum
15	3Sum	Medium	Sort + fix one, two-pointer rest
11	Container With Most Water	Medium	Move shorter wall inward
42	Trapping Rain Water	Hard	Track left_max & right_max
125	Valid Palindrome	Easy	Skip non-alphanumeric, compare
283	Move Zeroes	Easy	Partition: left=next valid position
26	Remove Duplicates from Sorted Array	Easy	slow tracks insertion point
19	Remove Nth Node from End (Linked List)	Medium	Fast leads by n steps

Pattern 3 — Hashing (HashMap / HashSet)

Hash structures give $O(1)$ average lookup, insert, and delete. They are the go-to when you need to remember 'what you've seen' or count frequencies. Almost every problem that requires fast lookup should make you think of a hash map first.

HashMap vs HashSet

Structure	Stores	Use When
HashMap (dict)	Key $\rightarrow$ Value pairs	You need to map something to something else (char $\rightarrow$ count, val $\rightarrow$ index)
HashSet (set)	Keys only	You need to check existence fast (seen before? duplicates?)

Common HashMap Patterns

Pattern	Template
Frequency count	<code>freq = {}; freq[x] = freq.get(x, 0) + 1</code>
Two Sum (index store)	<code>seen = {}; for i,v: if target-v in seen: return [seen[target-v], i]</code>
Anagram check	<code>Counter(s) == Counter(t)</code>
Subarray sum = k	<code>prefix_sum map; curr_sum - k in map $\rightarrow$ found</code>

LeetCode Problems — Hashing

#	Problem	Difficulty	Key Insight
1	Two Sum	Easy	Store val $\rightarrow$ index in map while iterating
242	Valid Anagram	Easy	Compare char frequency maps
49	Group Anagrams	Medium	sorted(word) as map key
128	Longest Consecutive Sequence	Medium	Set lookup; only start from chain heads
560	Subarray Sum Equals K	Medium	Prefix sum + frequency map
347	Top K Frequent Elements	Medium	HashMap + Heap or Bucket Sort
146	LRU Cache	Medium	HashMap + Doubly Linked List
290	Word Pattern	Easy	Bidirectional map

Pattern 4 — Recursion & Backtracking

Backtracking is a systematic brute-force search: you build a candidate solution step by step, and as soon as you detect a dead end, you 'undo' the last choice (backtrack) and try the next option. It's used when the problem asks for ALL valid solutions, not just one.

The 3-Step Mental Model

Step	Action	Code equivalent
Choose	Pick the next element to add	for choice in options:
Explore	Recurse with this choice made	backtrack(path + [choice], ...)
Unchoose	Remove choice, try next	path.pop() (or via function return)

Template

```
def backtrack(path, options):
    if is_valid_solution(path):
        result.append(path[:])
        return
    for choice in options:
        if is_valid(choice):
            path.append(choice) # Choose
            backtrack(path, new_options) # Explore
            path.pop() # Unchoose
```

LeetCode Problems — Backtracking

#	Problem	Difficulty	Key Insight
78	Subsets	Medium	Include or exclude each element
77	Combinations	Medium	Pick k from n, start index to avoid duplicates
46	Permutations	Medium	Use 'used' array to track picked elements
39	Combination Sum	Medium	Can reuse elements; no start+1
40	Combination Sum II	Medium	Sort + skip duplicates at same level
51	N-Queens	Hard	Check column, diagonal conflicts
79	Word Search	Medium	DFS on grid + mark visited
131	Palindrome Partitioning	Medium	Try all cut points, check palindrome

Pattern 5 — Divide & Conquer

Split the problem into smaller independent subproblems, solve each recursively, then combine results. The key is that subproblems don't overlap (if they do, use DP instead). This pattern is the foundation of Merge Sort, Quick Sort, and Binary Search.

Algorithm	Split	Combine	Complexity
Merge Sort	Half array each side	Merge two sorted halves	$O(n \log n)$
Quick Sort	Partition by pivot	No combine needed	$O(n \log n)$ avg
Binary Search	Discard half each step	No combine	$O(\log n)$
Max Subarray	Left half, right half, crossing	Max of 3	$O(n \log n)$

LeetCode Problems — Divide & Conquer

#	Problem	Difficulty
148	Sort List (Merge Sort on linked list)	Medium
23	Merge k Sorted Lists	Hard
53	Maximum Subarray (DC approach)	Medium
215	Kth Largest Element (Quickselect)	Medium
912	Sort an Array	Medium

Pattern 6 — Greedy Algorithm

Greedy makes the locally optimal choice at each step, hoping it leads to the globally optimal solution. It works when the problem has 'greedy choice property' — local optimum leads to global optimum. Proving this is hard; in interviews, trust the pattern if it fits and move on.

■ Greedy is the most dangerous pattern for beginners. It works on specific problems but FAILS on others. Always ask: 'Is the greedy choice provably optimal, or does a counterexample exist?'

LeetCode Problems — Greedy

#	Problem	Difficulty	Greedy Choice
455	Assign Cookies	Easy	Match smallest cookie to smallest child first
121	Best Time to Buy & Sell Stock	Easy	Track global min, update max profit
55	Jump Game	Medium	Track farthest reachable index
45	Jump Game II	Medium	Always jump to farthest in current range
134	Gas Station	Medium	Start from where total gas goes positive
435	Non-overlapping Intervals	Medium	Sort by end time; greedy removal
763	Partition Labels	Medium	Track last occurrence of each char
56	Merge Intervals	Medium	Sort by start; extend if overlap

Pattern 7 — Dynamic Programming (DP)

DP solves problems with overlapping subproblems and optimal substructure. You store the result of each subproblem so you don't recompute it. DP is the hardest pattern for most beginners and also the most frequently asked at top companies.

Two Approaches

Approach	Direction	When to Use
Top-Down (Memoization)	Recursive + cache (dict/array)	Easier to think about; implement recursion first
Bottom-Up (Tabulation)	Iterative, fill table from base case	More space efficient; used in optimal solutions

DP Problem Types (must know all)

Type	Classic Problem	LeetCode #
1D DP (linear)	Climbing Stairs, House Robber	70, 198
2D DP (grid)	Unique Paths, Minimum Path Sum	62, 64
0/1 Knapsack	Target Sum, Partition Equal Subset	494, 416
Unbounded Knapsack	Coin Change, Combination Sum IV	322, 377
LCS (Longest Common Subsequence)	Edit Distance, LCS	1143, 72
LIS (Longest Increasing Subsequence)	Russian Doll	300, 354
Interval DP	Burst Balloons, Matrix Chain	312
String DP	Palindromic Substrings, Wildcard Match	647, 44

Beginner DP Ladder (solve in this order)

#	Problem	Difficulty	Learn This
509	Fibonacci Number	Easy	Understand memoization vs tabulation
70	Climbing Stairs	Easy	$dp[i] = dp[i-1] + dp[i-2]$
198	House Robber	Medium	$dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$
213	House Robber II	Medium	Run 198 twice on sliced array
322	Coin Change	Medium	Classic unbounded knapsack
300	Longest Increasing Subsequence	Medium	$dp[i] = \max \text{ of all } dp[j] + 1 \text{ where } j < i$
1143	Longest Common Subsequence	Medium	2D DP: match or skip
416	Partition Equal Subset Sum	Medium	0/1 knapsack variant
72	Edit Distance	Hard	2D DP: insert/delete/replace costs
312	Burst Balloons	Hard	Interval DP (hardest type)

Pattern 8 — Binary Search

Binary Search doesn't just find elements in sorted arrays — it finds the boundary of a monotonic condition in $O(\log n)$. The 'search space' can be anything: indices, answer values, time, capacity.

Template (never get the boundary wrong again)

```
left, right = 0, n - 1 # or search space boundaries
while left <= right:
    mid = left + (right - left) // 2 # avoid overflow
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        left = mid + 1
    else:
        right = mid - 1
return -1 # not found
```

Binary Search on Answer (advanced, very common at Google)

When: 'Find minimum/maximum X such that condition holds.' The search space is the answer range, not the array indices. Define a feasibility function, binary search the answer.

LeetCode Problems — Binary Search

#	Problem	Difficulty	Type
704	Binary Search	Easy	Classic search
35	Search Insert Position	Easy	Find leftmost insertion
33	Search in Rotated Sorted Array	Medium	Determine which half is sorted
153	Find Minimum in Rotated Sorted Array	Medium	Binary search on rotation
74	Search a 2D Matrix	Medium	Treat 2D as 1D
875	Koko Eating Bananas	Medium	Binary search on speed (answer)
1011	Capacity to Ship Packages	Medium	Binary search on capacity
410	Split Array Largest Sum	Hard	Binary search on answer
4	Median of Two Sorted Arrays	Hard	Binary search on partition

Pattern 9 — Graph Traversal (BFS / DFS)

Graphs are nodes connected by edges. Trees are a special case of graphs (no cycles). Most graph problems use either BFS (level-by-level exploration) or DFS (deep exploration). The choice of BFS vs DFS changes the answer.

	BFS (Breadth-First)	DFS (Depth-First)
Data structure	Queue (FIFO)	Stack or Recursion
Best for	Shortest path, level order	Cycle detection, topological sort, backtracking
Space	O(width) — can be large	O(depth) — usually better
Implementation	from collections import deque	Recursive or explicit stack

BFS Template

```
from collections import deque
queue = deque([start])
visited = set([start])
while queue:
    node = queue.popleft()
    for neighbor in graph[node]:
        if neighbor not in visited:
            visited.add(neighbor)
            queue.append(neighbor)
```

DFS Template

```
def dfs(node, visited):
    visited.add(node)
    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(neighbor, visited)
```

LeetCode Problems — Graphs

#	Problem	Difficulty	Pattern
200	Number of Islands	Medium	DFS/BFS on grid
695	Max Area of Island	Medium	DFS + count cells
133	Clone Graph	Medium	BFS + HashMap
207	Course Schedule	Medium	Cycle detection (DFS/Topological)
210	Course Schedule II	Medium	Topological sort (Kahn's algo)
417	Pacific Atlantic Water Flow	Medium	Reverse BFS from both coasts
994	Rotting Oranges	Medium	Multi-source BFS
127	Word Ladder	Hard	BFS + character substitution
743	Network Delay Time	Medium	Dijkstra's algorithm

684	Redundant Connection	Medium	Union Find
-----	----------------------	--------	------------

Pattern 10 — Stack & Queue

Stack (LIFO) is used when you need to process things in reverse order, or when the most recent element is relevant. Queue (FIFO) is used in BFS and when order of processing matters.

Monotonic Stack (Very Common at Google)

A monotonic stack maintains elements in increasing or decreasing order. Used to find Next Greater Element, Previous Smaller Element, and area problems.

```
# Next Greater Element pattern
stack = [] # stores indices
result = [-1] * n
for i in range(n):
    while stack and arr[stack[-1]] < arr[i]:
        idx = stack.pop()
        result[idx] = arr[i]
    stack.append(i)
```

LeetCode Problems — Stack & Queue

#	Problem	Difficulty	Pattern
20	Valid Parentheses	Easy	Stack: push open, pop on close
155	Min Stack	Medium	Auxiliary stack for minimums
739	Daily Temperatures	Medium	Monotonic decreasing stack
496	Next Greater Element I	Easy	Stack + HashMap
84	Largest Rectangle in Histogram	Hard	Monotonic stack
232	Implement Queue using Stacks	Easy	Two stacks
225	Implement Stack using Queues	Easy	Two queues
239	Sliding Window Maximum	Hard	Deque (monotonic queue)
150	Evaluate Reverse Polish Notation	Medium	Stack for operands

Bonus Patterns

Prefix Sum

Build an array where $\text{prefix}[i] = \text{sum of arr}[0..i-1]$. Then $\text{range_sum}(l,r) = \text{prefix}[r+1] - \text{prefix}[l]$ in $O(1)$. Essential for any 'sum of subarray' question.

#	Problem	Difficulty
303	Range Sum Query - Immutable	Easy
304	Range Sum Query 2D	Medium
560	Subarray Sum Equals K	Medium
1248	Count Number of Nice Subarrays	Medium

Heap / Priority Queue

A heap gives $O(\log n)$ insert and $O(\log n)$ extract-min/max. Use when you need the k smallest/largest. Python's `heapq` is a min-heap by default; negate values for max-heap.

#	Problem	Difficulty
703	Kth Largest Element in a Stream	Easy
215	Kth Largest Element in an Array	Medium
347	Top K Frequent Elements	Medium
295	Find Median from Data Stream	Hard
23	Merge k Sorted Lists	Hard

Union-Find (Disjoint Set Union)

Tracks connected components efficiently. Each node points to a root. Union by rank + path compression gives nearly $O(1)$ per operation.

#	Problem	Difficulty
684	Redundant Connection	Medium
200	Number of Islands (alternate)	Medium
547	Number of Provinces	Medium
1584	Min Cost to Connect All Points	Medium

Bit Manipulation

#	Problem	Technique
136	Single Number	XOR cancels duplicates
191	Number of 1 Bits	$n \& (n-1)$ clears lowest bit
338	Counting Bits	DP + bit trick
190	Reverse Bits	Shift and OR

15-Day Google Interview Study Plan

■ Daily structure: 2 hours theory + pattern understanding | 2 hours solving problems | 30 min review + write out approach in plain English (simulates interview talk-through)

Day	Topic	Problems to Solve	Goal
1	Arrays + Sliding Window	121, 643, 3, 1004	Master fixed & variable window
2	Two Pointers	167, 125, 283, 11, 15	Sort → pointer logic
3	Hashing	1, 242, 49, 128, 560	Map patterns in all forms
4	Recursion Basics + Trees DFS	144, 94, 145, 104, 112	Tree DFS = recursion warmup
5	Tree BFS + Level Order	102, 107, 199, 116, 513	BFS queue pattern
6	Backtracking	78, 77, 46, 39, 131	Try-explore-undo
7	Binary Search	704, 35, 33, 875, 1011	Sorted + search on answer
8	Stack & Queue	20, 155, 739, 84, 150	Monotonic stack patterns
9	Greedy	455, 55, 45, 435, 763	Greedy choice proofs
10	DP Part 1 (1D)	509, 70, 198, 322, 300	State + transition
11	DP Part 2 (2D + LCS)	62, 64, 1143, 72, 416	Grid DP + string DP
12	Graph BFS/DFS	200, 133, 695, 994, 417	Grid graphs + island problems
13	Graph Advanced (Topo/Dijkstra)	207, 210, 743, 127, 684	Directed graphs
14	Mixed Review + Weak Areas	Redo 3-4 problems you struggled with	Identify + fix gaps
15	Full Mock Interview Day	2 random Medium problems, timed 45m	Simulate real interview

How to track progress

Status	Meaning	Action
■ Solved without help	You own this pattern	Review once more, move on
■ Solved with hints	Pattern recognized but gaps	Re-solve in 2 days without hints
■ Could not solve	Pattern not internalized	Study the editorial, implement, re-solve tomorrow

How to Respond in a Google Interview

Google interviews score you on: problem-solving process, coding ability, communication, and analytical thinking. The process is as important as the answer. An optimal solution with no talking often scores lower than a suboptimal solution with clear communication.

The Exact Talk-Through Script (memorize this structure)

Phase	What to Say	Time
1. Repeat & Clarify	"So if I understand correctly, I need to [restate problem]. Can I ask one [edge case]? What's the range of input?"	1-2 min
2. Brute Force	"The naive approach is $O(n^2)$ idea. For each element, I'd... but that's too slow for $n=10^5$."	1 min
3. Pattern Recognition	"I notice [keyword/property]. This looks like a [pattern] problem. Instead of [brute force], I can [optimize] with [idea]."	2-3 min
4. Walk Through Example	"Let me trace through the example: input=[2,7,11,15], target=9. Starting at left=0, right=3..."	2 min
5. Code (talk while coding)	"I'll initialize left=0 and right=n-1. In the while loop I'll check... I'm not sure if I should do this here because..."	10-15 min
6. Test & Debug	"Let me test with the example... edge case: empty array... what if all elements are the same?"	3-4 min
7. Complexity	"Time complexity is $O(n)$ because we visit each element once. Space is $O(1)$ since I only use two pointers."	1 min

Do This / Not That

DO	DON'T
Think out loud from the first second	Sit silently for 3+ minutes
Ask 1-2 clarifying questions before coding	Jump to code immediately
State brute force first, then optimize	Start with optimal and get confused
Write clean variable names (left, right, freq)	Use i, j, k, x for everything
Check edge cases explicitly at the end	Assume all inputs are valid
Say 'let me think about this' if stuck	Panic silently
Confirm with interviewer before optimizing	Optimize without communicating why

■ The single most important interview habit: After identifying the pattern, say your time complexity BEFORE you code. "I think this can be done in $O(n)$ using a sliding window — let me code that up." This shows you understand the problem, not just writing code randomly.

Curated LeetCode Problem List — Beginner to Advanced

Phase 1 — Foundations (Days 1-5, must solve all)

#	Problem	Pattern	Why It Matters
1	Two Sum	Hashing	Most asked problem ever; HashMap lookup
121	Best Time to Buy & Sell Stock	Greedy/Sliding	Track running min; Google classic
217	Contains Duplicate	Hashing	HashSet basics
242	Valid Anagram	Hashing	Frequency map
20	Valid Parentheses	Stack	Stack fundamental
704	Binary Search	Binary Search	Template practice
206	Reverse Linked List	Linked List	Pointer manipulation
104	Maximum Depth of Binary Tree	DFS	Tree recursion intro
226	Invert Binary Tree	DFS	Tree structure manipulation
283	Move Zeroes	Two Pointers	Partition pattern
70	Climbing Stairs	DP	Simplest DP problem
200	Number of Islands	Graph DFS/BFS	Grid graph fundamentals

Phase 2 — Core Patterns (Days 6-12, solve 70%+ of these)

#	Problem	Pattern	Difficulty
3	Longest Substring Without Repeating	Sliding Window	Medium
15	3Sum	Two Pointers	Medium
49	Group Anagrams	Hashing	Medium
198	House Robber	DP 1D	Medium
322	Coin Change	DP Knapsack	Medium
300	Longest Increasing Subsequence	DP	Medium
33	Search in Rotated Sorted Array	Binary Search	Medium
102	Binary Tree Level Order Traversal	BFS	Medium
207	Course Schedule	Graph/Topo	Medium
739	Daily Temperatures	Stack	Medium
55	Jump Game	Greedy	Medium
78	Subsets	Backtracking	Medium

46	Permutations	Backtracking	Medium
235	Lowest Common Ancestor of BST	Tree DFS	Medium
1143	Longest Common Subsequence	DP 2D	Medium
417	Pacific Atlantic Water Flow	Graph BFS	Medium
875	Koko Eating Bananas	Binary Search on Answer	Medium
347	Top K Frequent Elements	Heap + Hash	Medium

Phase 3 — Google-Level Hard (attempt if time permits)

#	Problem	Pattern	Note
84	Largest Rectangle in Histogram	Monotonic Stack	Stack masterclass
72	Edit Distance	DP 2D	Classic string DP
23	Merge k Sorted Lists	Heap / D&C	Heap mastery
127	Word Ladder	BFS	Graph + string manipulation
295	Find Median from Data Stream	Two Heaps	Design problem
312	Burst Balloons	Interval DP	Hardest DP type
4	Median of Two Sorted Arrays	Binary Search	$O(\log(m+n))$ solution
42	Trapping Rain Water	Two Pointers / Stack	Multiple solutions exist

Free Resources & References

Primary Resources (use these, not 10 others)

Resource	URL / Where	Best For
LeetCode	leetcode.com	Practice problems — main platform
NeetCode 150	neetcode.io	Curated 150 problems + video explanations for each
NeetCode YouTube	youtube.com/@NeetCode	Best free video explanations of every pattern
Striver's A2Z Sheet	takeuforward.org/strivers-a2z-dsa-course	Most comprehensive DSA roadmap for Indian students
CP-Algorithms	cp-algorithms.com	Deep theory: graphs, DP, number theory
Visualgo	visualgo.net	Visual animations of every algorithm
Google Interview Warmup	pgrow.google/certificates/interview-warmup	Official Google prep tool
LeetCode Discuss	leetcode.com/discuss	Real interview reports from Google candidates

Books (if you want depth)

Book	Author	Focus
Cracking the Coding Interview	Gayle McDowell	Interview process + 189 problems with explanations
Introduction to Algorithms (CLRS)	Cormen et al.	Deep theory — too heavy for 15 days, use as reference
Grokking Algorithms	Aditya Bhargava	Illustrated, beginner-friendly — read first if visual learner
Algorithm Design Manual	Skiena	Pattern-first approach, similar to this guide

YouTube Channels (ranked by quality for beginners)

Channel	Best Content
NeetCode	Pattern-based explanations, all major problems, clear code walkthroughs
Abdul Bari	Algorithm theory — best for understanding WHY, not just HOW
Back To Back SWE	Deep problem analysis, whiteboard-style
Tushar Roy	DP problems explained step by step
take U forward (Striver)	Hindi/English, very beginner friendly, popular in India

■ Final advice: Don't switch resources. Pick NeetCode + LeetCode and stick with them for all 15 days. Resource hopping is a procrastination trap. One source done completely beats five sources done partially.

Complexity Quick Reference

Time	When acceptable	Example algorithms
$O(1)$	Always	HashMap lookup, array access
$O(\log n)$	Always	Binary Search, Heap operations
$O(n)$	Best target for most problems	Sliding Window, Two Pointers, BFS/DFS
$O(n \log n)$	Usually fine	Merge Sort, sorting-based solutions
$O(n^2)$	Only if $n \leq 10^3$	Brute force DP, bubble sort
$O(2^n)$	Only if $n \leq 20$	Backtracking, brute force subsets
$O(n!)$	Only if $n \leq 10$	Brute force permutations

Best of luck for your Google interview. The goal is not to be perfect — it's to show clear thinking, communicate your process, and demonstrate that you can break down hard problems. That's all.